# Technical Supplement to OrchLang: Semantics, Finite Uncertainty Contracts and Reproducible Evidence

[Author names and affiliations to be supplied]

Candidate 0.5.0rc3; prepared 14 October 2026

**Code identity.** Immutable source revision: `8533214105aad0cc015ea1e1c2aad4057515aff7`. The main article and this supplement were written after the exact source revision passed its offline reproduction. Final TeX/PDF files and reproduction records are companion evidence; their hashes are bound by the publication index, not claimed to be part of that earlier code commit. Author declarations, independent human reproduction, provider calibration, persistent archive identity and submission remain pending.

**Abstract**

This supplement states the subjects of OrchLang's analyses, the normalized-IR refinement premises, and the finite-history row contracts used by two uncertainty prototypes. It derives the exact rectangular overlap recurrence, gives counterexamples to broader interpretations, and records experiment protocols, failures, caps and artifact custody. The mechanized result concerns paired normalized executions. The finite probability results concern supplied mathematical models. Execution experiments use synthetic offline oracles. None implies correctness of the extractor, real-provider probability calibration or provider billing.

## Contents

**1 Observation and semantic boundaries** **2**

**2 Normalized returned-state and graph refinement** **3**

**3 Conservative full-history row-ball bound** **3**

**4 Exact finite rectangular row-polytope distance** **5**

**5 Execution and source applicability studies** **7**

**6 Structured scaling and result reporting** **8**

**7 Prior art, novelty and external validity** **10**

**8 Artifact custody, reproduction and pending work** **11**

# 1 Observation and semantic boundaries

## 1.1 Abstract requests and accounting

The core language resolves a model alias to a contract $c$ containing provider/model identity, output cap and any declared price metadata. Lexical identities distinguish shadowed variables. A normalized call has exact decoded template bytes and ordered arguments: literals, fixed public values or positional references to prior matched call results. The abstract request is $Q = (t, \vec{v})$. A request-sensitive deterministic oracle is

$$\Omega(c, k, Q) = (r, u),$$

where $k$ is the contract invocation ordinal, $r$ a response and $u$ modeled output usage. Two executions with identical keys obtain the same pair. This is a coupling premise about the abstract oracle; SDK framing, tool schemas, hidden context, retry rules and provider wire serialization are absent from $Q$.

For density parameter $d \geq 1$, the additive input estimate is

$$\widehat{I}(t, \vec{v}) = \lceil |\operatorname{UTF8}(t)|/d \rceil + \sum_i \lceil |\operatorname{UTF8}(v_i)|/d \rceil .$$

Static bounds substitute declared argument bounds. Applying such a bound to an external tokenizer requires the per-call premise that every charged request component is covered and actual charged input is at most the derived expression. Equality of provider/model strings does not prove a tokenizer version or that premise. Model results reused across tokenizer identities are rejected by the compiler's compatibility check; no general cross-tokenizer conversion is inferred.

The invocation observer retains ordered $(c, Q, I, O)$ records. An ordered usage observer drops $Q$ and retains $(c, I, O)$; aggregate accounting further groups these labels by contract. Each projection can lose information. For example, swapping $(1, 3)$ and $(3, 1)$ produces equal totals but different ordered observations. Likewise, equal individual marginals do not identify a joint law: uniform laws on $\{00, 11\}$ and $\{01, 10\}$ have equal marginals and trace TV one. The artifact contains corresponding negative controls. The selected observer is part of every guarantee.

## 1.2 Closed finite-kernel trace equality

The existing graph analyzer constructs all reachable semantic states under a supplied finite rational success-outcome kernel and listed public/secret stores. A state retains program position, initialized values, result registers, per-contract ordinal saturated at 33, and the configured retained complete-event history. This finite closure is a semantic abstraction, not proof that a provider forgets its history or has saturated ordinal behavior.

For usage symbol $y$, let $A_y$ be its rational transition matrix. For two initial states, $\mu = e_0 - e_1$; $t_s, t_l$ are stopped and live indicator columns. For word $w = y_1 \cdots y_m$, differences are $\mu A_w t_s$ and $\mu A_w t_l$. Exact row-span exploration closes the space generated by $\mu A_w$. If all spanning rows annihilate both columns, every finite word has equal stopped/live mass. Otherwise the generating word supplies a finite witness. This is established weighted-automaton methodology [3, 4, 5]. It proves all-finite-horizon success-only equality in the supplied finite model; it does not establish infinite-run behavior or provider probabilities.

The exact replayer reconstructs the closed system, recomputes generator-word rows, checks independence, initial span, transition closure and both outputs. Unequal reports are checked by direct witness probabilities. Bisimulation is a separate sufficient analysis. The fixture `trace_equal_not_bisimilar` has equal usage laws but an inconclusive bisimulation result. The exact tool defaults to 10,000 states, 64 MiB deterministic byte-work, 10,000,000 matrix entries and 8,192 rational bits. Its meter is not RSS. Exhaustion yields UNKNOWN with no partial equality result. Source digests are custody fields, not proof that the recorded executable actually ran.

# 2 Normalized returned-state and graph refinement

## 2.1 Snapshot versus eager semantics

Source nodes read state fields from a fixed node-entry snapshot $S$, compute ordered local call results and return a field/local map $W$. The source atomically commits returned fields after the node completes. Target nodes use an eager store $T$, writing a produced returned local to its field as soon as the call completes. A later state-field read could therefore observe a different value in $T$ than in $S$.

The theorem `node_commit_refines_from` in `proofs/scp_strengthening/SequentialUpdates.v` assumes:

1. $S$ and $T$ initially agree at every text-field lookup;
2. `safe_after [] C W`: after a returned field is eagerly written, later calls do not read that field through a state-field atom;
3. call-result locals, returned fields and returned locals are unique;
4. every returned local is produced by the ordered call list.

Under these premises the resolved abstract request/response histories and local environments agree. The source's final atomic commit agrees field by field with the final eager store. The proof proceeds over calls, maintaining agreement for every field still read from the entry snapshot and agreement of prior local results. The shared oracle is an abstract total function of the modeled request history/request; usage and provider behavior are not part of this theorem.

Three machine-checked counterexamples explain the restrictions. A later read of a just-returned field changes the target request. Returning the same local to two fields leaves an eager register unwritten under the modeled lowering. Returning a field twice can disagree with eager write order. These are theorem-premise witnesses, not bugs claimed to be accepted by the current frontend.

## 2.2 Routed finite prefixes

`finite_prefix_graph_refines_including_cycles` uses separate lists of paired normalized workflow nodes. Calls, return maps and decisions in each pair agree; route targets use the same indices. Decisions are STOP, direct jump, immutable Boolean input or a text-field token comparison using any of $<, \leq, >, \geq, =, \neq$. After a node commits, equal states yield equal route decisions under the shared estimator. Induction on node-transition fuel therefore preserves abstract history, state lookup, status and program counter. Cycles, including a self-cycle, are permitted for every finite node prefix. No termination theorem follows, and node fuel does not model stopping in the middle of a call sequence.

Seven assumption reports are closed under the global context; no admitted goals or extra axioms occur. Local compilation used Rocq 9.3.0; the separate Ubuntu CI proof job also passed. Compiler simulation and composition are established methods [11]. This artifact adds a mechanized snapshot/commit distinction for OrchLang's normalized node model, not a proof of its complete compiler or source adapter.

# 3 Conservative full-history row-ball bound

Let $\Sigma$ be a declared complete finite observation alphabet containing a visible STOP symbol. At each scenario and complete visible prefix $h$ before the finite horizon $H$, the caller supplies nominal conditional row $q_i(h)$ and radius $\rho_i(h)$. The premise is

$$\mathrm{TV}(p_i(h), q_i(h)) \leq \rho_i(h)$$

| Mechanized subject | Bridge to executable/source behavior that remains unproved |
|---|---|
| Normalized field/local atoms | Python AST parsing, literal decoding, lexical/register names and JSON construction |
| Ordered calls and returns | Accepted source functions must construct the normalized records and satisfy safety/uniqueness/completeness |
| Paired route indices | Extracted graph entry/targets and implementation branching must match |
| Total text-field lookup | Coq uses a default empty value; backend raises on uninitialized reachable reads. Initialization is a separate obligation. |
| Abstract oracle and estimator | Actual request construction, runtime, errors, provider behavior and usage accounting |
| Finite completed-node fuel | Runtime caps can interrupt nodes; no theorem about partial eager states, retries or termination |

Table 1: These are explicit assumptions, not consequences of successful Coq compilation.

for the actual conditional row at every reachable history. Marginal constraints alone are insufficient. Missing live rows receive the maximally uncertain distance one. The caller must encode every selected status, usage or metadata field in the finite symbols, or define an explicit projection.

By triangle inequality the local paired distance is at most

$$d(h) = \min\{1, \mathrm{TV}(q_0(h), q_1(h)) + \rho_0(h) + \rho_1(h)\}.$$

Set $s(h) = 1$ at the horizon and after STOP. Otherwise, for the conservative possible common support $K(h)$,

$$s(h) = (1 - d(h)) \min_{y \in K(h)} s(hy).$$

When local distance is one the guarantee is zero; support/empty cases are handled conservatively. Maximal coupling matches the next symbol with probability at least $1 - d(h)$. Conditional on a match, the worst child continuation remains at least the displayed minimum. Induction yields a complete trace coupling agreeing with probability at least $s(\epsilon)$; the coupling inequality gives $\mathrm{TV}(P_0, P_1) \leq 1 - s(\epsilon)$ [10]. This uses standard coupling ingredients. The history-dependent fixture has bound 91/100, versus 99/100 from a coarser per-depth envelope.

Universal STOP-by-$H$ is a separate traversal over all possible events. If requested and verified, the bounded trace includes completed padded observations. Otherwise the certificate expressly covers the first $H$ slots and reports no termination proof. Errors are symbols, not implicit STOP. STOP padding begins at its first occurrence.

The prototype uses exact rationals and explicit caps: $H \leq 10{,}000$, 128 symbols of at most 256 UTF-8 bytes each, 50,000 explicit rows and at most 50,000 caller-selected traversal nodes, 16,000,000 document bytes, 1,000,000 JSON items with depth 64, 128-digit rational parts and 8,192-bit arithmetic. JSON integer inputs have separate digit and 512-bit guards. These maxima are acceptance limits, not demonstrated practical capacities; explicit prefix trees can grow rapidly. Replay traverses the certificate separately but shares parsing, arithmetic and the local obligation helper with the producer. A deliberate local-helper fault injection made both report a false zero distance for distinct deterministic symbols. This demonstrates common-code risk; no such defect was found in the current helper. The producer's output is checked against reader byte/tree caps before emission, and atomic output replacement occurs only after successful construction. The final suite has 23 tests. An independently constructed quarter-grid law enumeration checked 42,875, 117,649 and 15,435 kernel pairs; their maximal TVs were 1, 3/4 and 7/8, against bounds 1, 3/4 and 1. These 175,959 finite probes are falsification evidence, not proof over all real-valued rows.

# 4 Exact finite rectangular row-polytope distance

## 4.1 Contract and trace space

For scenario $i \in \{0, 1\}$ and full visible history $h$, the row set is

$$U_i(h) = \operatorname{conv}\{g_{i,h,1}, \ldots, g_{i,h,m}\},$$

where each $g$ is a rational distribution on the shared finite alphabet. A missing live row is the full simplex, represented by point masses. The contract is rectangular: each row in each scenario may be chosen independently of all other rows. No shared latent parameter, shared randomness or cross-scenario tie is represented. Finite alphabet completeness and conditional-history sufficiency are external premises.

Traces have exactly $H$ slots, padding with STOP after the first STOP. Without the optional universal stopping flag this is only a prefix law. If the flag is true, every positive-support path in both row sets must emit STOP by slot $H$. Errors, finish status and selected usage/metadata are visible ordinary symbols and do not implicitly terminate. The prototype does not infer any such field from an API.

For prefix $h$ and incoming masses $a, b \geq 0$, define

$$C_h(a, b) = \inf_{Q_0, Q_1} \sum_z \min\{aQ_0(z), bQ_1(z)\},$$

over allowed normalized suffix laws from that prefix. At $|h| = H$, $C_h(a, b) = \min(a, b)$. At a live prefix,

$$C_h(a, b) = \min_{g_0 \in G_0(h),\, g_1 \in G_1(h)} \left[ \sum_{y \neq \texttt{STOP}} C_{hy}(ag_0(y), bg_1(y)) + \min\{ag_0(\texttt{STOP}), bg_1(\texttt{STOP})\} \right]. \quad (1)$$

Here $G_i$ is the finite supplied generator list. Branches of zero mass contribute zero. The worst-case complete bounded TV is $1 - C_\epsilon(1, 1)$.

**Lemma 1.** *For any allowed suffix-law set independent of $a, b$, the function $C_h$ is concave and positively homogeneous in its two incoming masses.*

*Proof.* For fixed normalized laws, overlap is

$$\sum_z \min\{aQ_0(z), bQ_1(z)\} = \tfrac{1}{2}(a + b - \|aQ_0 - bQ_1\|_1).$$

Taking its infimum over all suffix pairs yields $\frac{1}{2}(a + b - D(a, b))$, with $D$ the supremum of convex weighted $L_1$ norms. The supremum is convex, so $C_h$ is concave. Scaling both masses scales each norm and overlap, hence positive homogeneity. The finite compact row contract attains its extrema; the same conclusion also follows by finite backward induction. □

**Proposition 1.** *Under the stated finite rectangular contract, recurrence* (1) *computes the exact minimum weighted overlap, and $1 - C_\epsilon(1, 1)$ is the maximum bounded trace TV.*

*Proof.* Partition traces by their next visible symbol. Overlap is the sum of child overlaps with masses $ap_0(y), bp_1(y)$. Rectangularity permits each child subtree's row choices to attain its minimum independently. STOP contributes its immediate overlap because subsequent padding is fixed. By the lemma each child value is concave, so their sum after linear mass scaling is concave in $(p_0, p_1)$. Every point in the product of the two row hulls is a convex combination of generator pairs. Concavity bounds its objective below by at least the minimum objective at a generator pair; since generator pairs are allowed, that minimum is exact. Induct from the horizon. Finally, normalized finite-law TV equals one minus overlap. □

This is a derivation for this finite software contract using established imprecise-probability, robust dynamic-programming and trace-distance ideas [6, 7, 8, 9]. No new general theorem, priority or unbounded exact-distance method is claimed. It also does not establish that an external process satisfies the contract.

### 4.2 Separating and scope counterexamples

The main article's singleton-row (fixed-kernel) half-mass fixture produces $AC$ and $BC$ in scenario 0, and $AD$ and $BC$ in scenario 1, each with probability 1/2. Its exact overlap is 1/2, TV 1/2, while the conservative worst-child continuation loses the matching $B$ mass and returns distance one. The recurrence carries both masses throughout arbitrary subproblems; their ratios can differ under uncertain rows. The fixture demonstrates a useful strict improvement, not empirical privacy protection.

Suppose a single shared parameter forces both scenarios to choose either $\delta_A$ or $\delta_B$. Each paired law has TV zero. Replacing the tied parameter by independently chosen row projections $\{\delta_A, \delta_B\}$ gives rectangular worst TV one. That value is exact for the product superset, not the original tied family. Similarly, dropping error/status fields, using unconditional marginals or omitting out-of-alphabet events changes the subject of the theorem. A row-set inclusion argument and an observer mapping are necessary to transfer its result.

### 4.3 Replay, caps and adversarial repairs

Work can grow exponentially with horizon, alphabet and generator choices; memoization and caps limit successful instances, without a polynomial complexity claim. The producer memoizes $(h, a, b)$, records its minimum and an attaining generator pair, and reserves a state before descent. Replay derives the complete closure under *all* generator pairs, recomputes local minima bottom-up and checks root/backpointer values. It never accepts a certificate solely because the recorded winning pair attains the claimed value. Parser, row lookup and rational arithmetic remain shared trusted code; the checker is not mechanized.

| Exact prototype limit | Value / behavior |
|---|---|
| Input/output document | 4,000,000 bytes input; output reserves one byte for newline |
| JSON | 200,000 items; nesting 64; strict UTF-8, duplicate-key/type/surrogate rejection |
| Horizon / alphabet | 10 / eight symbols; symbol length at most 128 UTF-8 bytes |
| Rows / generators | 20,000 rows; 32 generators per row; 100,000 total |
| DP / stop-check states | 30,000 each; unfinished states charged before recursion |
| Row-pair work | 2,000,000 charged symbol evaluations |
| Rationals | Input parts at most 48 digits; intermediate arithmetic 4,096 bits; bounded certificate encoding covers that intermediate domain |
| Failure result | Unverifiable error; no partial certificate or replacement of an earlier output |

Table 2: Caps delimit successful instances; they are not a performance theorem.

Fresh adversarial review found and repaired three exact-prototype defects: emitted tree size could exceed the reader's item cap; valid intermediate fractions could exceed the input rational digit limit reused by the certificate reader; and completed-state-only memo accounting omitted unfinished ancestors. The adjacent conservative prototype had the same output-item problem. An explicit depth-64 guard now avoids interpreter-dependent JSON nesting acceptance. Preserved reproducers cover each repair, including the one-state-cap boundary and a legal 93-digit intermediate fraction. The final exact suite has 21 tests.

The development whole-law reference shares the strict parser but enumerates complete vertex policies without the Bellman recurrence. Across 26 contracts it enumerated 201 left and 361 right

policies, 20,739 law pairs, with no exact/oracle mismatch or replay failure. A fresh separately implemented oracle uses neither subject parser nor row lookup: 120 contracts, 1,888 law pairs, 120 attaining vertex-policy witnesses, and four tamper rejections. These are bounded implementation checks, separate from the mathematical derivation and provider premise.

# 5 Execution and source applicability studies

## 5.1 Authored state differential protocol

The primary protocol and case/implementation hashes were recorded before execution. Eight authored workflows exercise multiple results, an internal temporary, overwrite of a returned field, and the six token-count comparisons. With the pinned actual runtime (LangGraph 1.2.14; LangChain Core 1.6.9) and a shared offline oracle, there are 96 completed and eight cutoff source/backend pairs and 224 events. Every pair checks ordered request, response, modeled usage, status and the completed source-state projection. Backend register names are projected to named source inputs/returned fields; internal temporaries are excluded. Format 2 returns null completed values on cutoff/runtime/resource failure.

For the three-call fixture with seed 17 and its recorded UTF-8 query, the returned draft, temporary critique and answer calls have units $(8, 5)$, $(9, 8)$ and $(15, 6)$. Totals are 32 input and 19 output units. The artifact retains exact query/response bytes and register mappings. This is a successful offline execution example, not a tokenizer receipt.

A later adversarial audit noticed the original equality comparison lacked a true-branch sample. A separate supplementary protocol was frozen before 18 new comparisons: all six operators at counts 1, 2 and 3. Both route outcomes occur for every operator. The initial supplement failed at preparation before executing source, due to noncanonical input encoding; that failure and original protocol are retained. The corrected canonical protocol passed 18/18. Four result mutations were detected, and an injected second-call error after an eager first write produced runtime error, one event and null completed state. Eight focused state-review tests pass. The final frozen run repeats the primary 104 and supplementary 18 comparisons into a fresh destination. These remain authored examples with a shared oracle, not general source-to-target correctness.

## 5.2 Immutable public-source execution capsules

The first capsule uses `didilili/ai-agents-from-zero`'s state/model-node and five exact wiring statements at commit `9de20be80dad6d410268ae75f75cf94c355dd364`. The second uses `JoshuaC215/agent-service-toolkit`'s three-node asynchronous knowledge-base agent at `88ebfa1fadd1a207aa92115635abf7ed20d66a88`. Exact AST source segments are compiled in a private module; the original whole module is not imported. Finite model/retriever/settings/logger doubles replace external boundaries. Actual graph/sequence execution is scoped to those selected statements and callables.

| Capsule | Declared finite support | Schedules | Trace classes | Answer classes |
|---|---|---|---|---|
| didilili | Two inputs × three model outcomes, weight 1/6 | 6 | 6 | 2 |
| JoshuaC215 | Two inputs × two retrieval outcomes × three model outcomes, weight 1/12 | 12 | 12 | 2 |

Table 3: Authored finite experimental laws, not estimates of deployment probabilities.

Both capsules record declared message types and IDs through LangChain Core's `message_to_dict` projection, together with response/usage metadata, finish reason, status, graph state, request envelope and events. Retrieval documents use a stated projection. The model support comprises two successful outcomes (stop/length with matching content) and a modeled runtime

error, so declared completion/error probabilities are 2/3 and 1/3. The second slice preserves its caught-retrieval-error behavior with an empty document list/logged failure. Content-only observation produces four classes in each capsule, collapsing distinctions kept by the full trace; final answers produce two. These are selected observer counterexamples, not a vulnerability attributed to the original repositories.

The macOS Seatbelt runner scrubs the environment; probes deny network, writes, fork and unlisted user files. Readability under `/opt` remains to launch the pinned Homebrew runtime, an explicit isolation limitation. The runner is host-specific and is not advertised as a portable general Python sandbox. No credentials or live provider keys were read.

The equally weighted schedule laws and their fixed observer projections are aggregated and replayed exactly from the hash-bound recorded traces. Bounded exploration and behavioral comparison have established predecessors [23, 24, 25]; the capsule contribution is the constrained integration of exact slices, declared schedules, offline doubles and observer-law replay.

The portable verifier rehashes source segments, worker/mock bytes, declared support, runtime identities, schedules, exact rational weights and repeated recorded traces. It imposes a fixed complete observer-field list, strict bounded JSON and source-path checks; 14 adversarial tests pass. It validates consistency of the recorded capsule, not that source execution happened as represented. The final reproduction replays these recorded models only. Neither capsule is a whole-file AST admission or a provider/SDK conformance test.

### 5.3 Disjoint held-out admission

The earlier development cohort has 12 repositories, 4,348 tracked Python paths, 59 graph candidates and zero admissions. The new protocol queries public Python LangGraph repositories, excludes development identities and infrastructure-only projects, and selects 12 eligible licensed repositories. Identities, commits, retrieval caps and source hashes were frozen before implementation inspection/evaluation. The extractor bytes stayed unchanged throughout this strengthening task.

Eleven repositories were retrieved; one exceeded the fixed 128 MiB archive cap and was retained as a retrieval failure, without replacement. The 1,580 retrieved Python paths contain nine AST graph candidates and 240 noncandidate parse errors. All nine candidates are unsupported: their first-rejection reasons are six unsupported imports and three top-level imports. Hence file admission is 0/9, while retrieval is 11/12. The parse errors and retrieval failure are not silently removed or counted as admitted/rejected candidates. No held-out source slices were attempted. The final reproduction verifies the frozen result bindings, without rescoring or executing third-party source. Zero admission precludes claims of representative framework support; the two earlier slices do not change this denominator.

## 6 Structured scaling and result reporting

### 6.1 Frozen design

The structured matrix contains 27 additional synthetic cases. Seven families vary secret fan-out, public guards, mutable registers, productive cycles, outcome support, observation alphabet and retained history. Two semantic contrasts distinguish exact equality/inequality from bisimulation; a resource probe deliberately reaches the state cap. Each current case has exact producer/replay and bisimulation producer/replay groups. Seven preregistered cases also compare pre-optimization revision 5aa4ebc's exact producer/replay. Thus there are $27 \times 4 + 7 \times 2 = 122$ groups, each with one warmup and five fresh measured workers: 732 raw, 610 measured rows.

No measured row in the normative run was an admission error, invalid result, worker error or timeout. The unequal history-dependent case has a two-observation witness with probability one versus zero, independently replayed. The equal-but-nonbisimilar case proves exact

| Measured operation/version | Counts across five repetitions per case |
|---|---|
| Current exact producer | 125 equal; five unequal; five UNKNOWN |
| Current exact replay | 130 verified; five UNKNOWN |
| Current bisimulation producer | 120 equal; ten inconclusive; five UNKNOWN |
| Current bisimulation replay | 130 verified; five UNKNOWN |
| Pre-optimization exact producer, seven cases | 30 equal; five unequal |
| Pre-optimization exact replay, seven cases | 35 verified |

Table 4: Inconclusive bisimulation and resource UNKNOWN are distinct classifications.

equality while bisimulation is inconclusive. All five repetitions of the resource case preserve UNKNOWN/STATE_CAP. Raw cap-stop times are positive diagnostic measurements; UNKNOWN contributes no completed-result latency and is never encoded as zero.

| Family | Reachable states | Producer median range (ms) | Replay median range (ms) |
|---|---|---|---|
| Secret fan-out | 4–16 | 0.83–2.38 | 0.90–3.65 |
| Public guards | 4 | 0.91–1.28 | 0.95–1.07 |
| Mutable registers | 10–190 | 1.89–77.53 | 2.47–164.82 |
| Productive cycles | 68–74 | 10.35–11.72 | 28.34–32.14 |
| Outcome support | 4–18 | 0.86–2.18 | 0.94–2.51 |
| Observation alphabet | 18 | 2.14–2.81 | 2.52–3.81 |
| Retained history | 16–46 | 2.72–11.57 | 3.51–10.09 |

Table 5: Ranges of per-case medians, five measured workers each; no pooled estimator.

The largest completed structured closure has 190 states and 316 transitions. Productive-cycle closure is finite because of the registered abstraction, including ordinal saturation. Structural changes alter multiple closure properties; these ranges do not isolate a state-count scaling law. Wall times are local macOS ARM64/Python 3.14.6 measurements. Deterministic work counters and frozen input bytes support repeatable comparisons; RSS, byte-work and elapsed time are distinct quantities.

### 6.2 Primary chains, parity and retained failures

The symmetric-chain baseline completes through 128 calls per arm in the current implementation, five of five repetitions. At 64 calls per arm, current replay median is 86.98 ms versus 138.77 ms at 5aa4ebc, with byte-work 1,784,025 versus 9,972,749. This meter reduction is not an RSS reduction. At 128, current analysis/replay medians are 82.83/320.02 ms and the old implementation is UNKNOWN, so no old completed-time comparison exists. At 256 both are UNKNOWN; current raw records retain BYTE_WORK_CAP, limit 67,108,864 and observed 67,116,168, with 514 states and 512 transitions. These are not completed latency samples.

The six relevant analysis modules are byte-identical to the rc2/dd97 source. That revision is a source-parity reference, not another timed implementation. The old 5aa version is the actual timed pre-optimization arm. Four earlier reporting attempts are retained: an input-hash binding issue, provisional summaries, mixed-version summaries and missing bisimulation work counters. The fifth structured result is normative. A validator exercises six known version-separated aggregation groups, checks all 122 real group repetitions, status/witness fields and saved summaries, and confirms UNKNOWN exclusion. The final 19-stage reproduction runs that validator, not 732 new timing workers.

# 7 Prior art, novelty and external validity

The focused search used OpenAlex discovery and primary publisher, author and repository records, checked 14 October 2026. It is neither systematic nor exhaustive and establishes no priority. The artifact retains the queries, source ledger and a longer claim-by-claim matrix.

| Primary work / status | Shared subject | Boundary relevant to OrchLang |
|---|---|---|
| RelCost, POPL 2017 [1]; resource-aware constant-time analysis [2] | Relational cost and resource-sensitive information flow | Established basis; OrchLang specializes modeled call/request/usage observers. |
| Rational automata [3, 4, 5] | Exact finite probabilistic language equality | Basis of row-span tool; no general algorithmic novelty. |
| Imprecise chains/TV ambiguity [6, 7]; labelled-chain TV [8] | Conditional uncertainty and trace distance | Basis of finite rectangular derivation; external row inclusion remains a premise. |
| PPDL, ICML 2026 [12] | Probabilistic LLM flows and uncertainty propagation | Adjacent objective is inference/uncertainty propagation, rather than the selected relational usage observer. |
| Lamina cost note [13]; typed-resources and Costwright [14, 15], author/software records | Pre-run workflow/token ceilings | Unary caps under declared increments/call bounds; they rule out broad novelty claims about static LLM budgeting. |
| MAT, accessible preprint and ENASE record [16] | Message-action traces, contracts, replay and perturbation | Neighboring assurance methodology; no priority claim for workflow trace replay. |
| Agentproof and CaMeLoT [17, 18], preprints | Extracted workflow graphs and temporal policy checks | Static agent verification is an active area; different properties and source bridges. |
| Guardians [19], CACM record and repository | Formal workflow verification | No full article-level technical comparison was undertaken; the comparison uses its record/repository. |
| APPA and FlowSeal [20, 21], preprints | Sensitive-content/provenance/tool-access flow | Adjacent confidentiality; selected observer differs without claiming their systems cannot influence usage. |
| ProMCP, Findings of ACL 2026 [22] | MCP pipeline token/latency measurement | Reinforces need to capture serialization overhead; empirical profiling does not prove relational confidentiality. |

The defensible software contribution is this bounded integration: request-sensitive paired call checking, supplied-kernel ordered-law decisions, finite row-contract distance checking, normalized snapshot/commit refinement and scoped execution evidence. No checked source was found asserting precisely this entire combination for this abstraction; this observation is not a firstness result.

The exact prototype's strict improvement is a useful artifact result. It is not evidence of a new general robust-Markov algorithm, a deployed privacy guarantee or a calibrated real-provider law.

Internal validity is supported by rational references, tamper/mutation probes, explicit caps, separate traversals and preserved failures. Shared parser/arithmetic/semantic helpers remain common failure points. External validity is constrained by synthetic kernels, two authored finite source-slice supports, zero held-out admission, one local timing environment and unproved source/provider bridges. Construct validity depends on the selected observer: aggregate counts, per-call usage, messages, stop/error status, latency and invoices are different objects.

# 8 Artifact custody, reproduction and pending work

## 8.1 Exact source and evidence identities

The final source commit is `8533214105aad0cc015ea1e1c2aad4057515aff7`. The source archive is generated by `git archive` of that revision. A source guard enumerates the committed tree and rejects differing executable/proof/config bytes and additional source outside explicit environments or byte-identical wheel copies. It includes root/build paths and rejects source symlinks. Runtime/compiler/proof-kernel/installed dependency integrity are environment premises, not proved host properties.

The packager requires all 19 stages in exact order with zero exit codes and matching source fingerprint. It hashes selected stage logs, binds the final TeX/PDF bytes to a recorded successful Tectonic build receipt, includes build logs and a claim/evidence index, checks archived source/evidence bytes and both ZIP CRCs. These are custody checks, not signatures or scientific-validity proofs. Tectonic/package-cache bytes are not hermetically pinned. Original source slices and their MIT/license notices are retained without whitespace normalization.

| Record | SHA-256 |
|---|---|
| Held-out source freeze | `e8eddcbb0779e0d66f6b9a9de9a21ed7f68a3980e1215b4b68fe3e817bc13fe6` |
| Unchanged extractor | `aa3c1cef891a40480b44dd438f137a5c5f91cc6b9285c38e3cc8bd5eed65f7d9` |
| Structured normative results | `b377646d3c2b49a5e64504f45cfa6c69cb8f2c2ef548af106e9a5636d1b4b70a` |
| dididili capsule payload | `1f8d8eca8860350d0d9d8422005c62dc222b5c79910d1c1a3d59fc61860e45dd` |
| JoshuaC215 capsule payload | `3b56bda34f906f4f44cf053316ee8b008119a55bcabb977f3b0b1095989342f1` |
| Public API plan | `9c8e2b720c5764b38b47c03cc3e824de131ebaf9abdff445370bb2a0727d2c42` |

Table 7: Payload identity is distinct from later certificate/replay metadata. Final archive hashes are in the companion index to avoid a self-referential PDF hash.

## 8.2 Fresh versus recorded stages

Fresh checks cover 112 native regressions, 230 audit tests, 22 frontend tests, 23 conservative and 21 exact tests, 19 fake-transport tests, 14 capsule tests, independent finite-law enumerations, 23 wheel validation stages, Coq compilation, pinned runtime identity, 11 backend/eight state-review tests and 104+18 authored comparisons. Recorded checks verify held-out custody, timing aggregates and prior capsule laws. The local final run passed all 19 stages and six CI jobs passed at the same code SHA: four Linux/macOS Python jobs, one Ubuntu Coq job and one pinned LangGraph job. Neither suite claims fresh paid API execution or independent human reproduction.

Three development reproduction attempts are preserved. The first ran audit tests before building the native binary; the second exposed interpreter-dependent deep JSON acceptance; the

third resolved a venv executable symlink to base Python without LangGraph. The stage order, explicit nesting guard and interpreter-path handling were repaired before freeze. These failed runs are not counted as passing evidence. The final frozen run uses its own passed manifest and fresh state destination.

From a checkout of the exact source, install the optional runtime using the supplied hash-locked requirements in `.venv`; provide make, a C++17 compiler, setuptools/wheel for offline packaging and `coqc`. Run:

```
python3 scripts/reproduce_scp_strengthening.py
-source-revision HEAD -native -package -coq
-langgraph-python .venv/bin/python
-output-dir output/scp_strengthening/faculty_replication
```

Use a fresh destination: the runner refuses to overwrite a passed frozen run, and state replication rejects populated output directories. The recorded scale rerun commands are in `resear ch/scp_strengthening/scalability/PREREGISTRATION.md`. The source-slice runner needs separate host-specific isolation review; ordinary reproduction never executes unchecked original modules. A blank faculty report form requests source/environment identity, commands, observed failures/differences and the researcher's confirmation. No report or contact is fabricated.

### 8.3 Provider calibration plan and publication status

The offline calibration harness freezes four public synthetic prompts under two mapped OpenAI Responses envelope profiles: eight response cases plus eight input-token preflights, at most 16 wire requests, 1,024 input and 64 output units per case, no tools, no retries/redirects and storage disabled. Provider/model and budget authorization are pending. A proposed $0.25 planning ceiling is not an authorized spend or guaranteed invoice cap. The runner is tested against fake transports and rejects missing usage fields rather than treating them as zero; cached/reasoning usage are retained when present. Tokenizer cache misses cannot trigger network fetches during offline validation.

These envelopes map selected source-shaped requests; they do not reproduce the original Gemini/AWS/provider SDK paths. A future eight-case sample would be descriptive calibration only, not a conditional-probability estimate, a statistically justified TV radius, future provider coverage or invoice validation. Such claims require a separate sampling/coverage design and the actual provider boundary.

SCP's Original Software Publication route is the primary target. The retained official template advises a short software paper and emphasizes evaluation of the software. SoftwareX is a plausible Elsevier fallback by subject fit; no submission-specific higher acceptance rate is established. The current journal guide must be checked by the authors before submission. Author/affiliation placeholders are intentional; correspondence, funding, conflicts, CRediT, originality/concurrent-submission facts, human approval and persistent archive identity require author completion. GPT-6 Luna agents at maximum reasoning assisted research and adversarial review, but agent reviews are not independent human evaluations. This package has not been submitted or accepted.

## References


[1] E. Çiçek et al., Relational cost analysis, POPL (2017). Publisher DOI.

[2] V. C. Ngo, M. Dehesa-Azuara, M. Fredrikson, J. Hoffmann, Verifying and synthesizing constant-resource implementations with types, IEEE S&P (2017), 710–728. Publisher DOI.

[3] W.-G. Tzeng, A polynomial-time algorithm for the equivalence of probabilistic automata, SIAM Journal on Computing 21(2) (1992), 216–227. doi:10.1137/0221017.

[4] S. Kiefer, A. S. Murawski, J. Ouaknine, B. Wachter, J. Worrell, Language equivalence for probabilistic automata, CAV (2011), 526–540. Publisher DOI.

[5] S. Kiefer et al., On the complexity of equivalence and minimisation for Q-weighted automata, LMCS 9(1) (2013). Publisher DOI.

[6] G. de Cooman, F. Hermans, E. Quaeghebeur, Imprecise Markov chains and their limit behaviour, PEIS (2009). Publisher DOI.

[7] I. Tzortzis, C. D. Charalambous, T. Charalambous, Dynamic programming subject to total variation distance ambiguity, SIAM J. Control Optim. (2015). Publisher DOI.

[8] T. Chen, S. Kiefer, On the total variation distance of labelled Markov chains, LICS (2014). Publisher DOI.

[9] G. Bian, A. Abate, On the relationship between bisimulation and trace equivalence in an approximate probabilistic context (2017). Author preprint.

[10] G. Barthe, T. Espitau, J. Hsu, T. Sato, P.-Y. Strub, *-Liftings for differential privacy, ICALP, LIPIcs 80 (2017), 102:1–102:12. Publisher DOI.

[11] X. Leroy, Formal verification of a realistic compiler, CACM 52(7) (2009), 107–115. Author manuscript.

[12] L. Mandel, G. Baudart, M. Vaziri, M. Hirzel, PPDL: LLM-Based Flows as Probabilistic Programs, ICML, PMLR 306 (2026), 86172–86197. Publisher record.

[13] K. Dickerson, Statically Bounding Resource Accumulation in LLM Agent Workflows: The Cost Case, author-hosted research note (18 May 2026). Primary note.

[14] H. Inverso, A Lean-Checked Potential Calculus for Bounded-Cost LLM-Agent Workflows: A Sequential Core, preprint v4 (12 June 2026), software revision 9ef2ee9. Version record; Pinned source.

[15] Costwright contributors, Costwright, pinned software revision 3d5212c (2026). Source record.

[16] C. Paduraru, P.-L. Bouruc, A. Stefanescu, A Trace-Based Assurance Framework for Agentic AI Orchestration: Contracts, Testing, and Governance, ENASE 21, vol. 1 (2026), 420–427. Publisher DOI; accessible preprint v1.

[17] M. Xavier, Vaisakh M A, M. Jolly, Midhun Xavier, Agentproof: Static Verification of Agent Workflow Graphs, arXiv:2603.20356v1 (20 March 2026).

[18] E. Nikolaou, M. W. Eckhoff, R. Flood, G. Grov, V. Mavroeidis, D. Aspinall, CaMeLoT: CaMeL orchestrated with Temporal logic for static verification and liveness, arXiv:2609.18674v1 (16 September 2026); record reports CAMLIS 2026 acceptance.

[19] E. Meijer, Guardians of the Agents: Formal Verification of AI Workflows, CACM 69(1) (January 2026), 46–52. Publisher record.

[20] A. Kravchenko, V. Liventsev, I. Konstantinov, I. Iskhakov, M. Kukuy, APPA: Recoverable Information-Flow Control for Real-World LLM Agents, arXiv:2607.24625v2 (26 August 2026).

[21] M. Shim, R. R. Karim, R. Jakkula, K. Zhou, X. Liu, X. E. Wang, Z. Li, Confuse the Model, Control the Flow: Understanding and Mitigating Privacy Leakage from LLM Agents with Information Flow Control, arXiv:2609.14003v1 (12 September 2026).

[22] S. Anjum, W. Zheng, R. Kettimuthu, H. Fan, Y. Feng, ProMCP: Profiling Token Flows and Latency Costs in Model Context Protocol–Based LLM Agents, Findings of ACL (2026), 39476–39487. Publisher DOI.

[23] E. Clarke, A. Biere, R. Raimi, Y. Zhu, Bounded model checking using satisfiability solving, FMSD (2001). Author manuscript.

[24] P. Godefroid, N. Klarlund, K. Sen, DART: Directed automated random testing, PLDI (2005). Author record.

[25] W. M. McKeeman, Differential testing for software, Digital Technical Journal 10(1) (1998). Publisher archive.